Coding Age

Mastering JavaScript for Interviews –Concepts , Code , & Examples

Day 1: JavaScript

1. JavaScript Basics

1. What is JavaScript, and why is it used?
2. Is JavaScript a compiled or interpreted language? Explain.
3. What type of language is JavaScript (programming vs scripting)?
4. What are the main features of JavaScript?
5. What are the differences between JavaScript and other scripting languages like Python or PHP?
6. Why is JavaScript considered dynamically typed?

---

2. JavaScript History

1. Who developed JavaScript and when?
2. What were the original names of JavaScript?
3. What's the difference between Java and JavaScript?
4. Why is JavaScript called JavaScript even though it's not related to Java?

---

3. Role in Web Development

1. What are the core technologies used in web development?
2. What is the role of JavaScript in web development?
3. How does JavaScript enhance user interactivity on web pages?
4. What is client-side scripting?
5. Can JavaScript be used for server-side programming? How?

---

4. Environment Setup

1. **How can you run JavaScript in a browser?**
2. **What is the browser console and how is it used?**
3. **What is the use of `console.log()` in JavaScript?**
4. **How do you write and run a JavaScript program inside an HTML file?**
5. **What tools/IDEs are commonly used to write JavaScript?**

---

5. Node.js

1. **What is Node.js?**
2. **How does Node.js allow JavaScript to run outside the browser?**
3. **What is the V8 engine in Node.js?**
4. **What are the advantages of using Node.js in web development?**
5. **Explain event-driven and non-blocking nature of Node.js.**
6. **What is NPM, and why is it important?**

---

6. Input/Output in JavaScript

1. **What is `prompt()` in JavaScript and when is it used?**
2. **What is `alert()` in JavaScript?**
3. **What is `console.log()` used for?**
4. **How do you take input from a user using JavaScript?**
5. **How do you convert a string to a number in JavaScript?**
   - Methods: `parseInt()`, `Number()`, unary `+` operator.

---

7. Java vs JavaScript

1. **What are the key differences between Java and JavaScript?**
2. **Which one is statically typed and which one is dynamically typed?**
3. **How does the execution of Java differ from JavaScript?**
4. **Where are Java and JavaScript typically used (use cases)?**

🔸 Bonus: Practical/Scenario-Based Questions

1. Write a JavaScript program to display "Hello, world!" in the console.
2. Write code to prompt the user for their name and greet them with an alert.

What will be the output of:

```
let num = +"10";
console.log(typeof num);
```

3. Why is `console.log()` useful during development?
4. What happens if a user cancels a `prompt()` dialog?

Day 2: Variables, Data Types, and Conversion

🔹 1. Variables

1. What is a variable in JavaScript and why is it used?
2. What is the difference between `var`, `let`, and `const`?
3. What do you mean by scope in JavaScript? Explain the scopes of `var`, `let`, and `const`.
4. Can you redeclare and reassign variables declared with `var`, `let`, and `const`?
5. What is the best practice when declaring variables? Why should `var` be avoided?
6. Give a practical example showing the difference between block scope and function scope.

🔹 2. Hoisting

1. What is hoisting in JavaScript?
2. How does hoisting behave differently with `var`, `let`, and `const`?
3. What is the "Temporal Dead Zone" in JavaScript?
4. What will be the output of the following code?

```
console.log(a);

var a = 5;
```

✅ Expected: `undefined`

5. **What will be the result of this? Why?**

```
console.log(b);
let b = 10;
```

❌ Expected: ReferenceError

🔹 3. Data Types

1. **What are the different data types in JavaScript?**
2. **Differentiate between primitive and non-primitive data types.**
3. **What is the difference between `null` and `undefined`?**
4. **What is a symbol in JavaScript and where is it used?**
5. **What is BigInt and when would you use it?**
6. **Is JavaScript a strongly typed language? Why or why not?**

🔹 4. ECMAScript

1. **What is ECMAScript, and how is it related to JavaScript?**
2. **Why was ECMAScript introduced?**
3. **Name some key features introduced in ES6.**
4. **What is the difference between ES5 and ES6?**
5. **How does ECMAScript help in browser compatibility?**

🔹 5. Type Conversion

✅ Implicit Conversion (Type Coercion)

1. **What is type coercion in JavaScript?**

---

2. **Explain with examples how JavaScript performs implicit type conversion.**
   - E.g., `"5" + 3` vs `"5" - 3`
3. **What will be the result of the following expressions?**

```
"10" + 5        // ?

"10" - 5        // ?
true + 1        // ?
null + 5        // ?
```

✅ Explicit Conversion

1. **How do you convert a string to a number in JavaScript? List all methods.**
2. **What is the difference between `parseInt()` and `Number()`?**
3. **How can you convert a number to a string?**
4. **How do you convert different values to Boolean in JavaScript?**
   - E.g., `Boolean(0)`, `Boolean("Hello")`, `Boolean("")`

🔸 Bonus Coding Questions

1. **Write a program to check the type of a variable and print its type using `typeof`.**
2. **Write code that shows how hoisting works with `var`.**
3. **Write a function that takes a string and returns its number equivalent. Handle cases where the input might be invalid.**
4. **Convert `"123.45"` to a number using different methods.**
5. **Write a code that demonstrates block scope using `let` and `const`.**

Day 3: Operators and Control Flow(Conditional and loops) and String.

🔹 1. Operators

---

1. What is the difference between `==` and `===`? Which one is preferred and why?
2. Explain the difference between `!=` and `!==` with examples.
3. What are truthy and falsy values in JavaScript? List all falsy values.
4. What is NaN in JavaScript? How do you check if a value is NaN?
5. Why does `typeof NaN` return `"number"`?

   What will be the result of:

```
console.log("5" + 2);   // ?
console.log("5" - 2);   // ?
```

6. What is short-circuit evaluation with `&&` and `||`?

   What is the output of:

```
console.log(true + false);
console.log("5" * "2");
console.log("five" * 2);
```

7. Difference between prefix and postfix `++` and `--` operators?
8. Explain the use of compound assignment operators like `+=`, `*=`, etc.

🔹 2. Control Flow (Conditionals)

1. What are different conditional statements in JavaScript?
2. When should you use a `switch` instead of an `if-else` ladder?
3. What is the ternary operator? Rewrite this using ternary:

```
if (score > 50) {

  result = "Pass";
} else {
  result = "Fail";
}
```

What is the output of:

```
if ("0") {
  console.log("True");
} else {
  console.log("False");
}
```

4. What happens if you forget to use `break` inside a `switch` case?
5. Can `switch` be used with string values? Provide an example.

---

3. Control Flow (Loops)

1. What are the different types of loops in JavaScript?
2. Explain the difference between `for`, `while`, and `do...while` loops.
3. What is the use of `break` and `continue`? Provide examples.
4. Write a loop to print all even numbers from 1 to 20.

What is the output of:

```
for (let i = 0; i < 5; i++) {
  if (i === 2) continue;
  console.log(i);
}
```

5. Write a program using a loop that stops when the user enters a negative number.

---

4. Strings

1. What is the difference between single quotes, double quotes, and backticks in JavaScript?
2. What is a template literal? How is it better than traditional string concatenation?
3. Write a program to reverse a string in JavaScript.
4. How do you check if a string contains another substring?
5. What is the difference between `slice()` and `substring()`?

---

6. **Write code to count the number of vowels in a given string.**
7. **How do you remove extra spaces from a string input?**
8. **How do you split a sentence into words using JavaScript?**

   What is the output of:

```
let text = "JavaScript";
console.log(text.indexOf("a"));
console.log(text.lastIndexOf("a"));
```

9. **List some commonly used string methods and their purposes.**

Day 4: Functions

🔹 Basic Conceptual Questions

1. **What is a function in JavaScript? Why do we use functions?**
2. **What is the difference between a function declaration and a function expression?**
3. **Explain the difference between parameters and arguments in a function.**
4. **What are default parameters? Give an example.**
5. **What is the purpose of the `return` statement in a function?**
6. **What happens if a function doesn't have a `return` statement?**

🔹 Arrow Functions

7. **What is an arrow function? How is it different from a normal function?**
8. **Can arrow functions use the `this` keyword? Why or why not?**
9. **Write an arrow function that returns the square of a number.**

🔹 Advanced / Real-World Usage

10. **What is a callback function? Can you give an example?**

---

11. **What is an IIFE (Immediately Invoked Function Expression)? Where is it used?**
12. **How can you return multiple values from a function in JavaScript?**
13. **Explain nested functions with an example.**
14. **What are higher-order functions? Can you give an example?**
15. **What is the difference between `call()`, `apply()`, and `bind()`?**

Practical Coding Questions (Asked in Technical Rounds)

16. Write a function to check if a number is **even or odd**.
17. Write a function that **reverses a string**.
18. Write a function that **calculates the factorial** of a number.
19. Write a function that **accepts another function as an argument** (callback example).
20. Write a function that returns the **maximum number in an array**.
21. Convert a regular function into an arrow function.
22. Write a function that returns `true` if a given string is a **palindrome**.

Day 5: Arrays, Array Methods , Spread and Rest Operator (...)

Core Array Concepts

1. **What is an array in JavaScript? How is it different from an object?**
2. **How do you create an array in JavaScript? Show two different ways.**
3. **What is the output of `typeof []` and why?**
4. **How do you find the length of an array?**
5. **How can you check if a variable is an array?**
   Example: `Array.isArray(variable)`

Accessing & Modifying Arrays

6. **How do you access and modify elements in an array?**
7. **What happens if you access an array index that doesn't exist?**

---

8. **How can you add or remove elements from the start and end of an array?**
   - `.push()`, `.pop()`, `.unshift()`, `.shift()`

🔹 Array Iteration Techniques

9. **What are the different ways to loop through an array in JavaScript?**
   - For loop, `for...of`, `for...in`, `forEach()`
10. **What's the difference between `for...of` and `for...in` when used with arrays?**
11. **How does `forEach()` work? Can it return a new array?**
12. **What is the difference between `.map()` and `.forEach()`?**
13. **How do you double every element in an array using `.map()`?**

🔹 Transforming Arrays

14. **How does `.filter()` work? Give an example.**
15. **What is `.reduce()` used for? Write a program to sum all values in an array.**
16. **What is the difference between `.map()`, `.filter()`, and `.reduce()`?**
17. **When would you use `.find()` vs `.filter()`?**
18. **What does `.splice()` do? How is it different from `.slice()`?**
19. **How does `.sort()` behave with numbers? Why does `[10, 2].sort()` return `[10, 2]`?**
20. **What does `.reverse()` do? Is it a mutating method?**

🔹 Spread and Rest Operators

21. **What is the spread operator (`...`)? List 2 use cases.**
22. **How do you merge two arrays using the spread operator?**
23. **How do you copy an array using the spread operator?**
24. **What is the rest parameter in a function? Give an example.**

---

25. **What is the difference between spread and rest operators?**

🔹 Advanced & Utility Methods

26. **What does `Array.from()` do? Give an example.**
27. **What is `Array.of()` and how is it different from `Array()` constructor?**
28. **What is `.flat()` in JavaScript arrays? How does it work?**

    Example: `[[1, 2], [3, 4]].flat()` → `[1, 2, 3, 4]`

29. **What is the result of `Array(3).map(x => 1)` and why?**
30. **How do you remove duplicates from an array?**

    Tip: Use `Set` → `Array.from(new Set(arr))`

🔹 Bonus Practical Coding Questions

These often appear in coding rounds or interviews:

31. Write a function that finds the **largest number** in an array.
32. Write a function to **reverse an array** without using `.reverse()`.
33. Write a function that removes **all falsy values** from an array.
34. Write a function that returns the **second largest number** in an array.
35. Flatten a nested array (1 level deep) using `.flat()`.

Day 6: Objects

🔹 Basic to Intermediate Object Questions

1. **What is an object in JavaScript? How is it different from an array?**
2. **How do you create an object in JavaScript? Mention two different ways.**
3. **How do you access and modify object properties using dot and bracket notation?**

---

4. **What happens if you access a non-existent key in an object?**
5. **What are nested objects? How do you access deeply nested properties?**
6. **What is `this` keyword inside an object method? Give an example.**
7. **Explain object destructuring with an example. How can you rename properties while destructuring?**
8. **What is the difference between shallow copy and deep copy of an object? Provide code examples.**

🔹 Practical Coding/Scenario-Based Questions

9. **Write a function that takes an object and logs all its keys and values.**
10. **Given an object, how would you count how many properties it has?**
11. **How do you iterate over an object's properties using `for...in` vs `Object.entries()`?**
12. **What's the output of the following code? Why?**

```
const obj = { a: 1 };
const copy = obj;
copy.a = 2;
console.log(obj.a); // ?
```

🔹 Advanced & Common in Interviews

13. **Explain prototype-based inheritance in JavaScript. How does the prototype chain work?**
14. **What is the difference between `Object.assign()` and spread operator for copying objects?**
15. **What is the difference between `Object.freeze()` and `Object.seal()`?**
16. **What are enumerable vs non-enumerable properties? How do they affect loops like `for...in`?**

17. **How do you merge two or more objects in JavaScript? What happens if they have the same keys?**
18. **What is the use of `Object.create()`? How is it different from object literals or constructors?**

🔹 MCQ/Tricky Conceptual Questions

19. **Which of the following methods creates a deep copy of an object?**

- a) `Object.assign()`
- b) Spread operator `...`
- c) `JSON.parse(JSON.stringify(obj))`
- d) All of the above

20. **What will the following log? Why?**

```
let user = {
  name: "Alice"
};
Object.freeze(user);
user.name = "Bob";
console.log(user.name); // ?
```

🔹 Bonus: Real-World Use Cases

21. **How would you represent a "Library" system using nested objects (Books, Authors)?**
22. **How do you check if a key exists in an object? (Hint: `in` operator and `hasOwnProperty`)**
23. **Write a function that flattens a nested object (1-level deep) into a single-level object.**

Day 7: Browser , Window Object , DOM (Document Object Model), BOM (Browser Object Model) , JavaScript Methods in Window

---

Mastering JavaScript for Interviews

🔹 1. Browser & Rendering

Q1. What happens when you enter a URL in the browser and press Enter?
(Common in Amazon, Flipkart, Infosys)

> A: DNS lookup → Server response → HTML fetched → DOM created → CSS/JS parsed → Page rendered.

Q2. What is the difference between rendering engines like Blink and Gecko?

Q3. Why is understanding how a browser works important in web development?

🔹 2. Window Object

Q4. What is the `window` object in JavaScript? How is it different from `document`?

> A: `window` is the global object; `document` is part of `window` that represents the DOM.

Q5. Is `let x = 5` added to the `window` object?

> A: No — only `var` declarations become properties of `window`.

Q6. What's the difference between `window.alert()` and `alert()`?

🔹 3. DOM (Document Object Model)

Q7. What is the DOM? How does JavaScript interact with it?

Q8. What are different DOM selection methods?
(Often asked in Capgemini, TCS coding tests)

> `getElementById`, `getElementsByClassName`, `querySelector`, `querySelectorAll`.

Q9. How do you change the style and content of an HTML element using JS?

Q10. How does event handling work in the DOM?

Explain `addEventListener()` and event bubbling.

🔹 4. BOM (Browser Object Model)

Q11. What is the BOM? How is it different from the DOM?

Q12. What are some important BOM objects?

`navigator`, `location`, `history`, `screen`, `window`.

Q13. How can you get the browser name and language in JavaScript?

Q14. What does `window.open()` do? Is it safe to use?

Q15. How would you redirect or reload the current page using BOM?

🔹 5. JavaScript Methods in Window

Q16. What is the difference between `setTimeout()` and `setInterval()`?

Common logic-based coding question.

Q17. How would you use `prompt()` to get user input and display it back on the page?

Q18. What are the different types of `console` methods? When should you use each?

Q19. What happens if you call `confirm()` in JavaScript? What values can it return?

🧠 Bonus: Real-World Application Questions

Q20. How would you dynamically update part of a webpage without refreshing the page?

Q21. Explain the lifecycle of a web page from request to render using DOM/BOM.

Q22. If you want to store global data accessible across all scripts, where would you store it?

A: `window` object.

Day 8: DOM Manipulation

Core DOM Concept Questions

1. **What is the DOM, and how is it different from HTML?**
2. **Explain the DOM tree structure. What are nodes in the DOM?**
3. **What is the difference between `innerText`, `textContent`, and `innerHTML`?**
4. **How is the DOM created by the browser? When does JavaScript get access to it?**

Element Selection Questions

5. **What are different ways to select elements in the DOM?**
   - Follow-up: What is the difference between `getElementById`, `querySelector`, and `querySelectorAll`?
6. **What is the difference between `HTMLCollection` and `NodeList`? Which one supports `forEach()`?**
7. **Why is `getElementsByClassName()` a live collection but `querySelectorAll()` is not?**

DOM Modification Questions

8. **How do you change the content of a DOM element using JavaScript?**
9. **What are the risks of using `innerHTML` to insert user-generated content?**

---

10. What's the difference between modifying styles via `style.color` vs adding a class with `classList`?
11. How do you dynamically change multiple styles at once using JavaScript?

DOM Attribute & Class Handling

12. How do you get or set an HTML attribute using JavaScript?
13. What is the difference between `className` and `classList`?
14. How does `classList.toggle()` work, and when is it useful?

DOM Traversal & Navigation

15. How do you find the parent, first child, or next sibling of an element in the DOM?
16. What is the difference between `childNodes` and `children`?

Element Creation & Removal

17. How do you dynamically create an HTML element and add it to the DOM?
18. What are the steps to append a new `<p>` tag with text to a div with id `container`?
19. How do you remove an element from the DOM using JavaScript?

Insert Before/After Questions

20. What's the difference between `append()`, `prepend()`, `before()`, and `after()` in JavaScript DOM manipulation?
21. How would you insert a new button before an existing `<div>` element using JavaScript?

Advanced & Real-World Questions

---

22. **How would you build a dynamic list where each new item is added on button click?**
23. **How can you toggle visibility of an element using JavaScript DOM methods?**
24. **What is event delegation and how does it relate to dynamic DOM elements?**

Day 9: Event Handling

🔹 Basic Event Handling

1. **What is event handling in JavaScript and why is it important in web development?**
2. **What are the different ways to handle events in JavaScript?**
   - Follow-up: Which is the best practice and why?
3. **What is the difference between using `onclick` in HTML vs `addEventListener` in JavaScript?**

🔹 Event Object

4. **What is the `event` object in JavaScript?**
   - Follow-up: Can you name and explain some useful properties of the `event` object?
5. **What is the difference between `event.target` and `this` inside an event handler?**
6. **What are `preventDefault()` and `stopPropagation()` used for?**
   - Follow-up: When would you use each one?

🔹 Mouse & Keyboard Events

7. **What are the commonly used mouse events in JavaScript?**
8. **What's the difference between `mouseover` and `mouseenter`?**
9. **How would you detect a specific key press like "Enter" in a form?**

---

10. **What's the difference between `keydown`, `keypress`, and `keyup` events?**

🔹 Event Delegation

11. **What is event delegation and why is it useful?**
    - Follow-up: How does it help with dynamic content?
12. **Can you explain how event delegation works internally with an example?**

🔹 Event Bubbling & Capturing

13. **What is event bubbling and event capturing in the DOM?**
14. **What are the differences between bubbling and capturing phases?**
15. **How do you make an event listener run during the capture phase?**
    - Follow-up: Give an example of when capturing is preferred.

🔹 Practical Scenarios

16. **How would you implement a button that disables itself after being clicked once?**
17. **How would you prevent a form from submitting if a required field is empty using event handling?**
18. **How would you dynamically attach and then remove an event listener?**
19. **How would you throttle or debounce event handlers like scroll or resize?**

🔹 Advanced Topics

20. **What are the pros and cons of using inline event handlers (`onclick="..."`) versus `addEventListener()`?**

---

21. **What are anonymous vs named functions in event handlers? Why might one be better for removal?**
22. **What happens if the same event listener is attached multiple times to the same element?**
23. **Can multiple event listeners of the same type be added to the same element? What happens?**

📌 Bonus Real-World Scenario Questions

1. **You have a list where new items are added via a form. How would you handle click events for delete buttons in each item?**
   → Expected answer: *Event delegation* on the parent container.
2. **Explain what would happen if you call `stopPropagation()` inside a bubbling event handler.**
3. **Why is it important to remove event listeners when they are no longer needed (e.g., in SPA or modal)?**

Day 10: OOPs (Object-Oriented Programming) Concept

✅ Basics and Key Concepts

1. **What is Object-Oriented Programming (OOP)? Explain its main principles.**
   (Encapsulation, Inheritance, Polymorphism, Abstraction)
2. **How does JavaScript support OOP if it's not a class-based language like Java?**
3. **What is the difference between a `class` and a `constructor function` in JavaScript?**
4. **What is the role of the `this` keyword in OOP? How does it behave in different contexts (arrow functions, global scope, methods)?**
5. **How is memory handled in constructor functions vs. using prototypes? Why are prototypes more efficient?**

✅ Encapsulation and Abstraction

6. **What is encapsulation in JavaScript? How can you implement it?**
7. **How do private properties work in ES6+ classes? What's the `#` symbol?**
8. **What is abstraction? How do you achieve it in JavaScript without abstract classes?**
9. **Can you access private fields outside the class? Why or why not? Show with code.**

✅ Inheritance

10. **What is inheritance in JavaScript? How is it implemented using `extends` and `super()`?**
11. **Why do we use `super()` in the child class constructor? What happens if we don't call it?**
12. **What's the difference between prototype-based and class-based inheritance in JavaScript?**
13. **Can you override a method from the parent class in the child class? How does it work?**

✅ Polymorphism

14. **What is polymorphism in JavaScript? How do method overriding and overloading differ?**
15. **Why is method overloading not natively supported in JavaScript? How can you simulate it?**
16. **Give an example of method overriding using ES6 classes.**

✅ Prototypes

17. **What is a prototype in JavaScript? How is it different from a class?**
18. **What's the difference between `__proto__` and `prototype`?**

---

19. How do you share methods among multiple objects efficiently in JavaScript?
20. What is the prototype chain? How does JavaScript use it for inheritance?

✅ Practical and Coding-Based Questions

21. Code: Create a `Vehicle` class and extend it with a `Car` class. Add methods to both. Instantiate and call the methods.
22. Code: Demonstrate encapsulation using private properties and methods in a class (using `#`).
23. Code: Create a `BankAccount` class that hides the balance using encapsulation and has a method to calculate interest.
24. Code: Simulate method overloading in JavaScript using default parameters or rest parameters.
25. Code: Show an example of prototype inheritance using `Object.setPrototypeOf()` or `__proto__`.

Day 11:Nested Function ,IIFE (Immediately Invoked Function Expression) .

Asynchronous JavaScript (Callbacks, Promises, and Async/Await)

🔁 Nested Functions & Closures

1. What is a nested function in JavaScript?
2. What is a closure, and how is it related to nested functions?
3. Why and when would you use a nested function in your code?
4. Can inner functions access variables of the outer function? Why?
5. Explain the concept of lexical scope with an example.
6. What is data encapsulation and how do nested functions help with it?
7. What will be the output of the following code? Why?

```
function outer() {
    let count = 0;
    return function inner() {
        count++;
        return count;
    };
}
const counter = outer();
console.log(counter()); // ?
console.log(counter()); // ?
```

⚡ IIFE (Immediately Invoked Function Expression)

8. **What is an IIFE? How is it different from a regular function?**
9. **What are the use cases of IIFE in JavaScript?**
10. **How does IIFE help in preventing global scope pollution?**
11. **Write an IIFE that logs a message immediately after it's defined.**
12. **Can an IIFE return a value or store data? Give an example.**

🧵 Synchronous vs Asynchronous JavaScript

13. **What is the difference between synchronous and asynchronous JavaScript?**
14. **Why is JavaScript considered single-threaded, and how does it handle asynchronous code?**
15. **What is the Event Loop in JavaScript?**

⏲ setTimeout & setInterval

16. **How does `setTimeout` work under the hood?**
17. **What's the difference between `setTimeout` and `setInterval`?**
18. **How can you cancel a `setTimeout` or `setInterval` operation?**
19. **What will be the output of this code and why?**

```
console.log("Start");
setTimeout(() => console.log("Inside setTimeout"),
0);
console.log("End");
```

🧩 Callbacks

20. **What is a callback function in JavaScript?**
21. **Explain callback hell with an example.**
22. **What are the drawbacks of using callbacks for asynchronous operations?**
23. **How can callback hell be avoided?**

🔗 Promises

24. **What is a Promise in JavaScript?**
25. **What are the three states of a Promise?**
26. **How do you handle errors in Promises?**
27. **Explain Promise chaining with a real-world example.**
28. **How do Promises solve the problem of callback hell?**
29. **What is the difference between `Promise.all`, `Promise.any`, and `Promise.race`?** (asked in advanced rounds)

🔄 Async/Await

30. **What is async/await in JavaScript?**
31. **How does async/await improve code readability?**
32. **What happens if you forget to use `await` in an async function?**
33. **How do you handle errors in async/await code?**
34. **Convert the following Promise-based code to async/await:**

```
fetchData()
```

```
    .then(data => process(data))
    .catch(error => console.log(error));
```

🧠 Conceptual + Practical

35. **How would you implement a delay function using Promises?**
36. **Write a function that logs numbers 1 to 5 with a 1-second delay between each log using:**

- `setTimeout` + Callbacks
- Promises
- Async/Await

37. **What's the difference between `await Promise.all([...])` and multiple `await` statements?**
38. **How is asynchronous behavior achieved in a single-threaded environment like JavaScript?**

Day 12: Fetch API, JSON, and HTTP

🔹 Fetch API & Async/Await

1. What is the Fetch API in JavaScript?
2. How does the Fetch API differ from XMLHttpRequest?
3. What does the `fetch()` method return?
4. How do you handle errors when using fetch()?
5. How do you check for a failed HTTP response in fetch()?
6. How do you use async/await with fetch()?
7. What are the advantages of using async/await over `.then()` chaining?
8. What is the role of `response.ok` in fetch?
9. Can you give an example of chaining multiple fetch requests?
10. What is the difference between `.then()` and `await`?

🔹 JSON (JavaScript Object Notation)

11. What is JSON and why is it used in web development?

---

12. What is the difference between `JSON.parse()` and `.json()`?
13. How do you convert a JavaScript object to a JSON string?
14. How do you convert a JSON string into a JavaScript object?
15. What are some common use cases of JSON in real-world applications?

🔹 HTTP & Status Codes

16. What is HTTP and how does it work?
17. What are the most commonly used HTTP request methods?
18. What is the difference between GET and POST requests?
19. What does a 404 status code mean?
20. What are the differences between 401, 403, and 500 status codes?
21. What status code is returned when data is successfully created?
22. How do you handle different HTTP status codes in fetch?

🔹 Real-World & Project-Based

23. How would you fetch data from an API and display it on a web page?
24. How would you use fetch() to send a POST request with a JSON payload?
25. How would you handle multiple API requests sequentially and display the combined result?
26. What are CORS errors and how do they relate to fetch()?
27. How can you add headers (like Content-Type or Authorization) in a fetch request?
28. What are some common mistakes developers make when working with fetch and how can they be avoided?

Day 13: Advanced Concepts – Closures, Lexical Scoping, Call, Apply, Bind, and Local Storage and Date and Time

---

Mastering JavaScript for Interviews

🔹 Closures & Lexical Scoping

1. What is lexical scoping in JavaScript?
2. What is a closure in JavaScript?
3. Can you give a real-world use case of closures?
4. How does JavaScript maintain access to outer variables in closures?
5. What are some practical applications of closures in web development?
6. How do closures help in data privacy or encapsulation?
7. What will be the output of a closure-based counter function?
8. Explain how garbage collection works with closures.

🔹 Call, Apply, and Bind

9. What is the difference between `call()`, `apply()`, and `bind()` in JavaScript?
10. When would you prefer `apply()` over `call()`?
11. What does `bind()` return and how is it different from `call()`?
12. Give an example where `bind()` is useful in an event listener.
13. How does `this` behave differently in arrow functions vs regular functions with `call`/`bind`?
14. Can you polyfill the `bind()` method?

🔹 Local Storage

15. What is `localStorage` in JavaScript?
16. How is `localStorage` different from `sessionStorage` and cookies?
17. How do you store and retrieve data from localStorage?
18. Can `localStorage` store objects? If so, how?
19. What are the storage limits of `localStorage`?
20. How secure is the data stored in localStorage?
21. What happens to localStorage data on browser refresh or tab close?

🔹 Date and Time

22. How do you create a new date in JavaScript?
23. How can you get the current year, month, and date?
24. Why does `getMonth()` return 0 for January?
25. What are the methods to format a date into a readable string?
26. How do you get the current timestamp in JavaScript?
27. How can you calculate the difference between two dates?
28. How do you convert a date to ISO format or local time string?